A DECo is an Unbiased Estimator of PseCo

Proof. In this section, we provide the detailed proof that the proposed DECo loss $\mathcal{L}_{\text{DECo}}$ is an unbiased estimator of the PseCo loss $\mathcal{L}_{\text{PseCo}}$. We begin by defining the expected PseCo loss over the joint distribution $P(X, Y)$:

$$\tilde{\mathcal{L}}_{\text{PseCo}} = \mathbb{E}_{(\boldsymbol{x},y)\sim P(X,Y)} \left[l_{\text{PseCo}}(\boldsymbol{x}_i, y_i) \right] \tag{15}$$

By rearranging the terms to separate the class-wise sample means, we obtain:

$$l_{\text{PseCo}}(\boldsymbol{x}_i, y_i) = -\log \left\{ \frac{\frac{1}{N_{y_i}} \sum_{p\in A(y_i)} e^{\boldsymbol{z}_i \cdot \boldsymbol{z}_p / T}}{\sum_{j=1}^{K} \frac{N_j}{N} \left(\frac{1}{N_j} \sum_{a\in A(j)} e^{\boldsymbol{z}_i \cdot \boldsymbol{z}_a / T} \right)} \right\} \tag{16}$$

As the total number of samples $N \to \infty$, by the law of large numbers, the sample means converge to the class-conditional expectations. For the numerator, where features $\boldsymbol{z}_{y_i}$ follow the vMF distribution of class y_i:

$$\frac{1}{N_{y_i}} \sum_{p\in A(y_i)} e^{\boldsymbol{z}_i \cdot \boldsymbol{z}_p / T} \xrightarrow{N\to\infty} \mathbb{E}_{\boldsymbol{z}_{y_i}\sim \text{vMF}(\boldsymbol{\mu}_{y_i}, \kappa_{y_i})} \left[e^{(\boldsymbol{z}_i/T)^\top \boldsymbol{z}_{y_i}} \right] \tag{17}$$

Similarly, for the denominator, letting π_j denote the prior probability of class j:

$$\begin{aligned} \frac{1}{N} \sum_{j=1}^{K} \sum_{a\in A(j)} e^{\boldsymbol{z}_i \cdot \boldsymbol{z}_a / T} &= \sum_{j=1}^{K} \frac{N_j}{N} \left(\frac{1}{N_j} \sum_{a\in A(j)} e^{\boldsymbol{z}_i \cdot \boldsymbol{z}_a / T} \right) \\ &\xrightarrow{N\to\infty} \sum_{j=1}^{K} \pi_j \mathbb{E}_{\boldsymbol{z}_j\sim \text{vMF}(\boldsymbol{\mu}_j, \kappa_j)} \left[e^{(\boldsymbol{z}_i/T)^\top \boldsymbol{z}_j} \right] \end{aligned} \tag{18}$$

Combining these results, the expected loss becomes:

$$\tilde{\mathcal{L}}_{\text{PseCo}} = \mathbb{E}_{(\boldsymbol{x},y)\sim P(X,Y)} \left[-\log \left\{ \frac{\mathbb{E}_{\boldsymbol{z}_{y_i}\sim \text{vMF}} \left[e^{(\boldsymbol{z}_i/T)^\top \boldsymbol{z}_{y_i}} \right]}{\sum_{j=1}^{K} \pi_j \mathbb{E}_{\boldsymbol{z}_j\sim \text{vMF}} \left[e^{(\boldsymbol{z}_i/T)^\top \boldsymbol{z}_j} \right]} \right\} \right] \tag{19}$$

To compute the analytical form, we evaluate the expectation of the exponential inner product over the hypersphere S^{d-1} using the vMF density function $f(\boldsymbol{z}_j; \boldsymbol{\mu}_j, \kappa_j) = C_d(\kappa_j) e^{\kappa_j \boldsymbol{\mu}_j^\top \boldsymbol{z}_j}$:

$$\begin{aligned} \mathbb{E} \left[e^{(\boldsymbol{z}_i/T)^\top \boldsymbol{z}_j} \right] &= \int_{S^{d-1}} e^{(\boldsymbol{z}_i/T)^\top \boldsymbol{z}_j} f(\boldsymbol{z}_j; \boldsymbol{\mu}_j, \kappa_j) d\boldsymbol{z}_j \\ &= \int_{S^{d-1}} C_d(\kappa_j) e^{(\boldsymbol{z}_i/T + \kappa_j \boldsymbol{\mu}_j)^\top \boldsymbol{z}_j} d\boldsymbol{z}_j \end{aligned} \tag{20}$$

Let $\kappa'_j = \|\kappa_j \boldsymbol{\mu}_j + \boldsymbol{z}_i/T\|_2$, where κ'_j is the concentration parameter of the resulting unnormalized distribution and the updated mean direction $\boldsymbol{\mu}'_j = (\kappa_j \boldsymbol{\mu}_j + \boldsymbol{z}_i/T)/\kappa'_j$,. The integral simplifies to:

$$\begin{aligned} \mathbb{E} \left[e^{(\boldsymbol{z}_i/T)^\top \boldsymbol{z}_j} \right] &= C_d(\kappa_j) \int_{S^{d-1}} e^{\kappa'_j \boldsymbol{\mu}'^\top_j \boldsymbol{z}_j} d\boldsymbol{z}_j \\ &= \frac{C_d(\kappa_j)}{C_d(\kappa'_j)} \end{aligned} \tag{21}$$

Finally, substituting this back into the loss expression, we arrive at the analytical form of the DECo loss:

$$\tilde{\mathcal{L}}_{\text{PseCo}} = \mathbb{E}_{(\boldsymbol{x},y)\sim P(X,Y)} \left[-\log \frac{\pi_{y_i} C_d(\kappa_{y_i}) / C_d(\kappa'_{y_i})}{\sum_{j=1}^{K} \pi_j C_d(\kappa_j) / C_d(\kappa'_j)} \right] = \tilde{\mathcal{L}}_{\text{DECo}} \tag{22}$$

□

B Computation of Normalization Factor

To compute the normalization factor $C_d(\kappa)$ within the von Mises-Fisher (vMF) distribution, we first establish its mathematical role in ensuring that the probability density function integrates to unity over the unit hypersphere $\mathbb{S}^{d-1}$. The factor is formally defined as:

$$C_d(\kappa) = \frac{\kappa^{d/2-1}}{(2\pi)^{d/2} I_{d/2-1}(\kappa)} \tag{23}$$

where $I_{d/2-1}(\kappa)$ denotes the modified Bessel function of the first kind at order $d/2-1$ defined as:

$$I_{(d/2-1)}(z) = \sum_{k=0}^{\infty} \frac{1}{k!\Gamma(d/2-1+k+1)} \left(\frac{z}{2}\right)^{2k+d/2-1} \tag{24}$$

While this definition is mathematically precise, the direct calculation of high-order Bessel functions introduces a significant computational bottleneck. Specifically, forward recurrence relations are notoriously prone to numerical instability when the concentration parameter κ is not sufficiently large.

To circumvent these stability issues, the DEACON framework adopts the Miller recurrence algorithm, which utilizes an inverse recurrence relation to determine the function values. The relation is expressed as:

$$\hat{I}_{\nu-1}(\kappa) = \frac{2\nu}{\kappa}\hat{I}_{\nu}(\kappa) + \hat{I}_{\nu+1}(\kappa) \tag{25}$$

where $\hat{I}$ represents trial values in the sequence. The process begins by initializing trial values at the higher end of the order, specifically setting $\hat{I}_d(\kappa) = 1$ and $\hat{I}_{d+1}(\kappa) = 0$, and then iteratively computing the values backward for $\nu = d-1, d-2, \ldots, 0$. The precise value for the required higher-order function is then derived by scaling the trial values according to:

$$I_{d/2-1}(\kappa) = \frac{I_0(\kappa)}{\hat{I}_0(\kappa)}\hat{I}_{d/2-1}(\kappa) \tag{26}$$

where $\hat{I}_0(\kappa)$ and $\hat{I}_{d/2-1}(\kappa)$ are the trial values obtained through the Miller recurrence, and $I_0(\kappa)$ can be efficiently computed using PyTorch.

C Details of Data Augmentation

To construct two distinct views for Self-SimSiam training, we apply Weak Augmentation and Strong Augmentation to the same input images. This multi-view generation strategy aims to enhance the feature extractor's robustness and consistency across varying levels of perturbation.

Weak Augmentation. The weak augmentation view, denoted as $\tilde{x}_w$, is generated via standard geometric transformations designed to introduce moderate spatial shifts while preserving the core semantic information of the image. The augmentation pipeline follows a series of stochastic transformations:

- Random Horizontal Flip
- Random Crop

Strong Augmentation. The strong augmentation view, denoted as $\tilde{x}_s$, is generated using the RandAugment strategy [Cubuk *et al.*, 2020]. This approach significantly increases the diversity of the training data by applying a series of complex transformations. The augmentation process is defined as follows:

- Stochastic Transformation Selection
- Dynamic Magnitude Sampling
- Random Cutout

D Mixup

MixUp is a simple and data-agnostic data augmentation method that generates mixture samples by mixing two images of different classes. For a pair of two images and their labels probabilities (x_i, p_i) and (x_j, p_j), we calculate $(\tilde{x}, \tilde{p})$ by:

$$\begin{aligned} &\sigma \sim Beta(\zeta, \zeta), \\ &\tilde{x} = \sigma x_i + (1-\sigma)x_j, \\ &\tilde{p} = \sigma p_i + (1-\sigma)p_j. \end{aligned} \tag{27}$$

where σ is sampled from a $Beta$ distribution parameterized by the ζ hyper-parameter. For our method, we fixed the mixup hyperparameter $\zeta = 4$, $\sigma = 0.6$. Specifically, we construct new training samples by linearly interpolating a sample $x_i \in D_{rel}^{head}/x_i \in D_{rel}^{tail}$ with another sample $x_j \in D_{rel}^{head}/x_j \in D_{rel}^{tail}$ randomly: $x_i^m = \phi x_i + (1-\sigma)x_j$, $q_i^m = \phi q_i + (1-\sigma)q_j$. Then we define mixup loss as the cross-entropy of predictions of x^m and q^m:

$$\mathcal{L}_{\text{mixup}}^{\text{tail}} = \frac{1}{|\mathcal{D}_{rel}^{tail}|} \sum\nolimits_{x_i \in \mathcal{D}_{rel}^{tail}} \mathcal{H}(\boldsymbol{p}_i^{mt}, \boldsymbol{q}_i^{mt}) \tag{28}$$

$$\mathcal{L}_{\text{mixup}}^{\text{head}} = \frac{1}{|\mathcal{D}_{rel}^{head}|} \sum\nolimits_{x_i \in \mathcal{D}_{rel}^{head}} \mathcal{H}(\boldsymbol{p}_i^{mh}, \boldsymbol{q}_i^{mh}) \tag{29}$$

where $p_i^{mh} = \text{softmax}(g(f(\boldsymbol{x}_i^m; \theta^{head}); \boldsymbol{W}^{head}))$ denotes the head-dominant branch predication for $\boldsymbol{x}_i^m$; $p_i^{mt} = \text{softmax}(g(f(\boldsymbol{x}_i^m; \theta^{tail}); \boldsymbol{W}^{tail}))$ denotes the tail-focused branch predication for $\boldsymbol{x}_i^m$.

E Complexity Analysis

Let P, B, D, and K denote the dimension of the input sample, the batch size, the dimensionality of the feature, and the number of classes. Let H denote the hidden dimensionality of the model. Our time complexity is $O(B(PH + HD + DK))$. We show the running time of DEACON and baselines (seconds/epoch) in Table 5, demonstrating DEACON achieves a runtime within the same order of magnitude as the baselines.

Method		CIFAR10-LT	CIFAR100-LT
PLL	LW	2.78	2.49
	CC	2.62	2.57
	PRODEN	3.47	2.83
	CAVL	5.49	4.82
	PiCO	10.57	10.34
LT-PLL	RECORDS	16.23	15.83
	SoLar	7.23	7.27
	HTC	6.78	7.16
	Ours	13.67	13.44

Table 5: Running time (seconds/epoch) of DEACON and baselines on CIFAR10-LT ($\eta = 0.5, \gamma = 250$), CIFAR100-LT ($\eta = 0.1, \gamma = 150$).

F Supplementary ablation results

After performing an ablation study on the Dual-Branch architecture, distribution-aware contrastive learning module, and self-SimSiam training module in the main body of the paper, there are still several components in our framework. The supplementary ablation study is shown in Table 6. Replacing the ensemble with a single branch significantly degrades performance, with overall accuracy dropping to 77.42% (Tail-Focused) and 75.22% (Head-Dominant). Mutual Disambiguation is one of the most important modules; its removal causes an overall performance drop to 62.32%. Furthermore, the removal of mixup leads to a consistent performance decline across all metrics, while the removal of logit adjustment results in a slight reduction.

Method	CIFAR10-LT			
	Overall	Many	Medium	Few
DEACON(Ours)	85.78	89.82	84.50	81.67
with Only Tail-Focused Branch	77.42	92.38	79.80	47.77
with Only Head-Dominant Branch	75.22	69.50	80.47	84.93
w/o Logit Adjustment	84.95	87.88	83.50	82.50
w/o Mixup	82.65	90.30	79.73	75.37
w/o Mutual Disambiguation	62.32	83.25	73.30	23.43

Table 6: Supplementary ablation results on CIFAR10-LT($\eta = 0.5, \gamma = 100$)